

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

CI Checks

All submissions must pass the automated Agent checks. This reference explains what each check validates, which Dockerfile checks block by default, which ones warn by default, and how to fix common failures.

Running Agents Locally

BASH

 Copy

```
# Using GPT-5.5 (recommended - matches CI)
stb harbor run -m @openai/gpt-5.5 -p <task-folder>
```

For a pre-submission static pass, run:

BASH

 Copy

```
harbor tasks check <task-folder> -m openai/@openai/gpt-5.5
```

Structural Checks

Some checks can report structural errors even if the related rule normally warns. For example, missing `environment/`, missing `environment/Dockerfile`, missing `tests/test.sh`, or malformed `task.toml` can surface as errors because the checker cannot safely inspect the task.

validate_task_fields

What it checks: All required fields are present in `task.toml`.

How to fix: Ensure `task.toml` has the required `version`, `[metadata]`, `[agent]`, `[verifier]`, and `[environment]` fields for your task type. Milestone tasks must use the milestone-specific layout and `[[steps]]` configuration.

check_task_absolute_path

What it checks: Task instructions use absolute paths.

MARKDOWN

 Copy

```
# Bad
Edit config/settings.json

# Good
Edit /app/config/settings.json
```

check_privileged_containers

What it checks: No privileged containers or unsafe Docker capabilities are used in `docker-compose.yaml`.

YAML

 Copy

```
# Bad
privileged: true
```

check_task_sizes

What it checks: Individual task files stay within platform limits. Larger runtime datasets are handled separately by `check_build_context_size`.

How to fix: Remove, split, compress, or fetch optional large data at runtime from an approved mounted source instead of baking it directly into the submitted task.

Dependency And Image Checks

pinned_dependencies

What it checks: Language-package dependencies use exact version pins.

DOCKERFILE

 Copy

```
# Bad
RUN pip install numpy pandas

# Good
RUN pip install numpy==1.26.4 pandas==2.1.0
```

Package-manager lockfiles are also acceptable where appropriate, such as [package-lock.json](#), [pnpm-lock.yaml](#), [uv.lock](#), [poetry.lock](#), [Cargo.lock](#), [go.sum](#), or Maven/Gradle dependency locks.

check_pinned_images

Severity: Blocking by default.

What it checks: Every Docker [FROM](#) image is pinned by digest. Tags are useful for readability, but the digest is the immutable pin.

DOCKERFILE

 Copy

```
# Bad
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm

# Good
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256
```

Apply the same discipline to service images in [docker-compose.yaml](#) when a service uses [image:](#) instead of [build:](#).

check_sanctioned_base_images

Severity: Blocking by default.

What it checks: The final runtime stage uses a [canonical Terminal-Bench base image](#). Builder stages may use task-appropriate toolchain images, but the final stage must land on a canonical base — or a non-canonical base accompanied by a brief, credible justification in the Dockerfile or task [README.md](#).

See the [canonical base image list](#) for the full set (Python, Node, Go, Rust, Java, Ruby, GCC, Maven, Debian, Ubuntu).

DOCKERFILE

[Copy](#)

```
# Bad: final runtime stage is not canonical (and no justification p
FROM hexpm/elixir:1.16@sha256:<digest>

# Good: final runtime stage is a canonical base
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256
```

If you need a non-canonical final runtime base, include a brief justification and flag it for review before submission.

check_reproducible_builds

Severity: Warning by default.

What it checks: Dockerfiles avoid nondeterministic downloads and package installs.

DOCKERFILE

[Copy](#)

```
# Bad
RUN curl -LsSf https://astral.sh/uv/install.sh | sh

# Good
ARG UV_VERSION=0.6.14
ARG UV_SHA256=<sha256>
RUN curl -fsSL "https://github.com/astral-sh/uv/releases/download/$
    && echo "${UV_SHA256} /tmp/uv.tar.gz" | sha256sum -c - \
```

```
&& tar -xzf /tmp/uv.tar.gz -C /usr/local/bin --strip-components  
&& rm /tmp/uv.tar.gz
```

Dockerfile Layout And Hygiene Checks

check_build_context_size

Severity: Blocking by default.

What it checks: The `environment/` build context stays lazy-pull friendly: at most 100 MiB total and at most 50 MiB per file.

DOCKERFILE

 Copy

```
# Bad  
COPY huge_dataset/ /app/data/  
  
# Good  
COPY fetch_seed.py /app/fetch_seed.py
```

Keep only small required seed data in the image. Mount or fetch large optional data at runtime only when the task design and platform allow it.

check_dockerignore

Severity: Warning by default.

What it checks: Non-trivial `environment/` directories include a `.dockerignore`.

Recommended entries:

DOCKERIGNORE

 Copy

```
.git  
**/__pycache__/  

```

```
*/*.pyc
*/node_modules/
.env
solution/
tests/
```

check_dockerfile_hygiene

Severity: Warning by default.

What it checks: The build context and image do not include local clutter or secrets.

Remove or ignore:

- `environment/.git/`
- `environment/.env`
- `environment/**/*.pycache/`
- `environment/**/*.node_modules/`
- editor files, caches, logs, credentials, and unused build outputs

check_apt_usage

Severity: Warning by default.

What it checks: Debian/Ubuntu package installation follows one clean apt transaction per stage and avoids upgrades.

DOCKERFILE

 Copy

```
# Bad
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get upgrade -y

# Good
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

check_layer_volatility

Severity: Warning by default.

What it checks: Dockerfile layers are ordered from least volatile to most volatile so dependency layers can be cached.

DOCKERFILE

[Copy](#)

```
# Bad
COPY . /app
RUN pip install -r /app/requirements.txt

# Good
COPY requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt
COPY src/ /app/src/
```

check_no_build_tools_in_runtime

Severity: Warning by default.

What it checks: Compilers and build tools are not left in the final runtime image unless the task explicitly requires the agent to use them.

DOCKERFILE

[Copy](#)

```
# Bad: final stage compiles and keeps build tools
RUN apt-get update && apt-get install -y build-essential
RUN make

# Good: compile in a builder, copy artifact into slim runtime
FROM public.ecr.aws/docker/library/rust:1.85-slim@sha256:<digest> A
RUN cargo build --release --locked

FROM public.ecr.aws/docker/library/debian:bookworm-slim@sha256:<dig
COPY --from=builder /build/target/release/tool /usr/local/bin/tool
```

check_file_extraction

Severity: Warning by default.

What it checks: Archives copied into the image are extracted and removed in the same stage.

DOCKERFILE Copy

```
# Bad
COPY fixtures.tar.gz /tmp/fixtures.tar.gz

# Good
COPY fixtures.tar.gz /tmp/fixtures.tar.gz
RUN mkdir -p /app/fixtures \
    && tar -xzf /tmp/fixtures.tar.gz -C /app/fixtures \
    && rm /tmp/fixtures.tar.gz
```

check_heredoc_usage

Severity: Warning by default.

What it checks: Dockerfiles do not embed source files through heredocs.

DOCKERFILE Copy

```
# Bad
RUN cat > /app/foo.py <<'EOF'
print("hello")
EOF

# Good
COPY foo.py /app/foo.py
```

check_recursive_permissions

Severity: Warning by default.

What it checks: Dockerfiles avoid broad recursive metadata rewrites.

DOCKERFILE Copy

```
# Bad
RUN chmod -R 755 /app
RUN chown -R app:app /app

# Good
COPY --chmod=0755 run.sh /usr/local/bin/run-task
COPY --chown=app:app src/ /app/src/
```

Runtime And Verifier Checks

tests_or_solution_in_image

What it checks: The `tests/` folder and `solution/` files are not copied into the Docker image.

DOCKERFILE

[Copy](#)

```
# Bad
COPY tests/ /tests/
COPY solution/ /solution/
```

check_dockerfile_references

What it checks: Dockerfiles do not reference forbidden solution or verifier files.

Remove references such as:

- `solution/solve.sh`
- `tests/test.sh`
- `test_outputs.py`

check_test_sh

What it checks: `tests/test.sh` runs the Python pytest verifier, produces a reward file, and uses dependencies that were baked into the image. For non-

Python tasks, the pytest file should call the application or service under test rather than replacing pytest with another test runner.

BASH

 Copy

```
#!/bin/bash
set -uo pipefail

mkdir -p /logs/verifier

python -m pytest /tests/test_outputs.py -rA
rc=$?

if [ "$rc" -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

check_offline_tests

Severity: Warning by default.

What it checks: `tests/test.sh` does not install packages or download from the network.

BASH

 Copy

```
# Bad
pip install requests
npm install
curl https://example.com/fixture.json
git clone https://github.com/example/repo.git

# Good: dependencies are baked into the image
python -m pytest /tests/test_outputs.py -rA

# Good: local-only install from preloaded wheels
pip install --no-index -f /opt/wheels pytest==8.4.1
```

Other Checks

typos

What it checks: Spelling errors in file and variable names.

How to fix: Review flagged items and correct spelling.

ruff

What it checks: Python code passes linting.

BASHCopy

```
ruff check <task-folder>
ruff check --fix <task-folder>
```

Quick Reference Table

CHECK	DEFAULT RESULT	WHAT IT VALIDATES
<code>check_pinned_images</code>	Blocks	Every <code>FROM</code> image
<code>check_sanctioned_base_images</code>	Blocks	Final runtime base
<code>check_build_context_size</code>	Blocks	<code>environment/</code> <=
<code>pinned_dependencies</code>	Blocks	Language deps ha
<code>tests_or_solution_in_image</code>	Blocks	No tests/solution in
<code>check_dockerfile_references</code>	Blocks	No forbidden solut
<code>check_test_sh</code>	Blocks	Reward file written
<code>check_task_absolute_path</code>	Blocks	Instructions use ab

CHECK	DEFAULT RESULT	WHAT IT VALIDATES
<code>check_privileged_containers</code>	Blocks	No privileged mode
<code>validate_task_fields</code>	Blocks	Required TOML fie
<code>ruff</code>	Blocks	Python linting pass
<code>typos</code>	Blocks	Names are spelled
<code>check_task_sizes</code>	Blocks	Files stay within pl
<code>check_dockerignore</code>	Warns	Non-trivial context
<code>check_dockerfile_hygiene</code>	Warns	No context clutter
<code>check_offline_tests</code>	Warns	Verifier is network-
<code>check_apt_usage</code>	Warns	Apt usage is clean
<code>check_reproducible_builds</code>	Warns	Downloads are pin
<code>check_layer_volatility</code>	Warns	Cache-friendly laye
<code>check_no_build_tools_in_runtime</code>	Warns	Runtime image is s
<code>check_file_extraction</code>	Warns	Archives extracted
<code>check_heredoc_usage</code>	Warns	Source files are no
<code>check_recursive_permissions</code>	Warns	No broad <code>chmod -</code>

Next Steps

- [Dockerfile & Image Best Practices](#) - Detailed image guidance
- [Quality Guidelines](#) - Additional quality standards
- [Review LLMAJ checks](#)
- [Submit your task](#)

PREVIOUS

[CI Feedback Training](#)

NEXT

[LLMaJ Checks Reference](#)